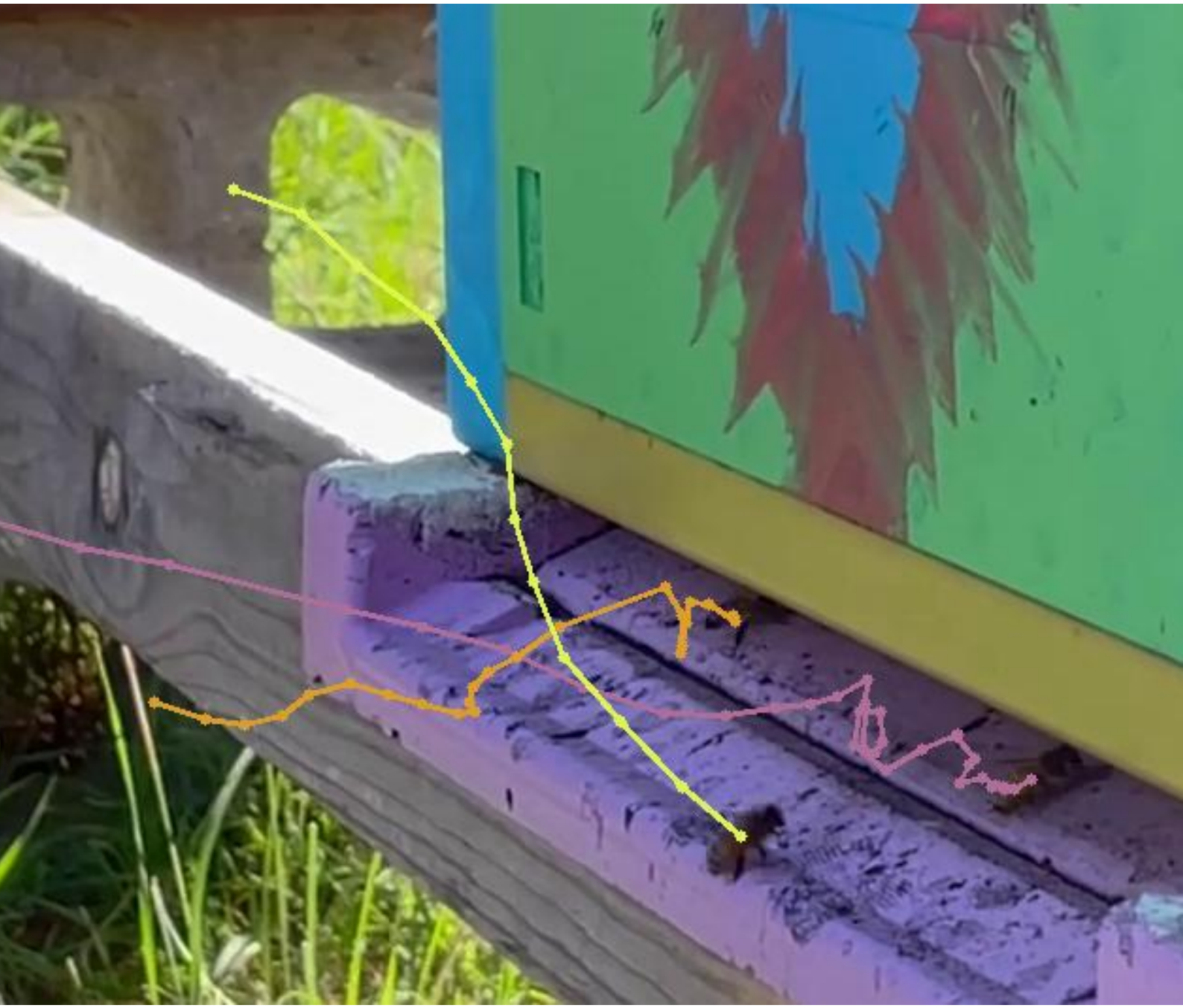
Hive Cam:

Developing a Model to Track Honeybee Foraging Behaviour

By
Toby Davis

October 2025

Supervisors
Jing Zhang
Alexander Mikheyev



Abstract

Honeybees are such a crucial part of our ecosystems, our agriculture, and our food security. And while technological developments in the precision agriculture sphere have allowed us to understand much more about their day-to-day behaviour, as well as their responses to threats such as climate, pests, and environmental changes, there is still a big need for innovative ways to monitor bee colonies autonomously. Most of the work in this space has prioritised the hive interior, which makes sense, as this is where most of the colony activities take place. However, there is a rich untapped wealth of understanding to be yielded from simply observing the hive entrance. From here, we can understand bee foraging patterns, monitor food intakes into the colony, and see the state of internal activities by observing what bees do on the landing pad. In recent years there has been a heightened interest in the entrance way as a place to monitor bee activity, and we want to step further into this space. Here, we propose and develop a new model to track honeybees in flight as they enter and exit the hive entrance, and then to count these numbers of bees over time. We do this by modelling the tracking problem as a conditionalized linear sum assignment optimisation problem, which is informed by a unique set of tracking logic specific to bee behaviour. We found that this model was able to robustly track bees in most cases, being able to account for missed detections and changing numbers of bees in the frame. Our counts of bees entering and exiting the hive over time have proven to be accurate to 92%. There is more work to be done to finetune and develop this model further to improve performance and add additional features, but the core of the Hive Cam project was a success, and is freely available to be downloaded to test for anyone who is interested. We hope to pick this work up in a subsequent project and develop a more complete system for beekeepers and researchers to use around the country, and indeed, the world.

Table of Contents

Hive Cam:
Developing a Model to Track Honeybee Foraging Behaviour

Toby Davis, with Dr. Jing Zhang and Prof. Sasha Mikheyev (supervisors)

Check out the project page to download the desktop app and explore the code:
https://github.com/TobysWorkshop/Hive-Cam/

1 Introduction

The world of the honeybee is a complex and fascinating one.

The hive-minded insects that communicate through dance (Dong et al., 2023), detect magnetic fields with their stomachs (Liang et al., 2016), engage in democracy (Seeley, 2010), and for whatever reason have decided to pick an age-old battle with wasps (Pusceddu, et al., 2017). Because of all this complexity, there is still a lot that we don't know or fully understand about these social pollinators. This means that there are still lots of opportunities to explore in the intersection of *technology* with apiculture.

We want to step further into this space.

Particularly, we want to step further into the external hive monitoring space – a field we believe to be underappreciated when it comes to understanding and monitoring hive health.

We propose the development of an app that can act like a beehive doorbell camera (a hive-cam, if you will). The idea is that this app will track the foraging behaviour of the colony by counting the number of honeybees entering and exiting the hive entrance over time, identifying the role of counted bees, and then displaying this information to the user. If we can deploy this app for citizen science use, then this information can be pooled from colonies around the country to help researchers understand how honeybee colonies are being affected by environmental changes. This could then further be used by beekeepers to identify optimal times for drone trapping.

In this report we outline our progress in getting the core aspects of this idea working. This includes a new model and tracking logic that targets this specific application, and the beginnings of an app for public use. Before we begin to delve into the design and technical aspects of the project, however, it will help if we start by wrapping our heads around some of the context for this problem.

1.1 Honeybee foraging behaviour

Honeybees (*Apis mellifera*) live and forage as colonies, often containing between 10,000 and 80,000 honeybees (AHBIC, 2022). In honeybee society, the female *worker* bees perform most of the day-to-day work throughout the colony – from maintaining the hive walls to feeding larvae (AHBIC, 2022). However, possibly their most crucial (and, indeed, for honeybees, distinctive) role is foraging for nectar and pollen from flowers outside the hive. This provides food, nutrition, and supplies for winter months.

Because foraging is such a crucial activity, the hive entrance becomes one of the most valuable places for understanding the health of a hive and its colony. Beekeeper Susan Chernak McElroy goes so far as to say that "most everything I need to know about my bees, I learn by sitting in front of my hives" (Chernak McElroy, 2016). The hive entrance allows bees to regulate temperature (which they do by flapping their wings to generate airflow), transfer resources from *foragers* (workers that leave the hive) to *house workers* (workers that stay in the hive), and defend against predators.

Because of this, healthy hives are often entrance-busy hives. If more bees are coming and going, or hanging out on the landing platform, then chances are the colony is going strong. You can also tell a lot about a colony's internal activities from the entrance – are lots of bees fanning at the entrance? It's likely honey is being processed. Are bees beginning to cluster at the entrance in a beard-like shape? They may be getting ready to swarm (Chernak McElroy, 2016).

For this reason, most man-made hives have a large slit entrance in the front face of the box, such as the one pictured in Figure 1.

Figure 1: *Photo of a typical bottom hive entrance (Williams, 2025)*

Worker bees that go out to forage will return to this entrance in an airstrip which we will call *the runway* – a stretch of about one metre in front of the hive entrance where foragers line up their flight and descend to land on the entrance platform, before crawling into the inner hive (authors' observation).

If we want to understand our hives better, we need to understand their entrances better.

1.2 The need for a robust, easy-to-use entrance-monitoring model

While there are a number of existing bee-monitoring technology applications, there is a great need for more robust, easy-to-use, and non-intrusive methods.

For instance, plenty of models for hardware-based bee counting have been proposed, such as the one by Pešović et al. (Pešović et al. 2017). These models rely on physical barriers being set up in the entrance way that only allow one bee to pass through at a time. A laser or camera can then be used to count them as they pass between it and the base of the barrier. While effective, these systems require extensive modification of the beehive itself, which can be costly and time-consuming for beekeepers.

Models that focus on the hive entrance are also not super common in mainstream monitoring systems. The more common approach is to monitor the inside of the hive using temperature, weight, and humidity sensors (see, for instance: The Urban Beehive, 2025).

This is why we want to develop a computer vision system that focuses on observing the hive entrance, and that is robust enough to be deployed in a variety of contexts. We hope that this is the sort of non-intrusive information that can tell both beekeepers and researchers more about their colonies, and colonies around the country, respectively.

2 The Object Detection Model

The first step in solving this problem was to train an object detection model to recognise honeybees in still images. For this, we used two sources of training data: one set of videos taken at the beehives at the Australian National University (ANU), where our research is based, and another set obtained from the Mendeley Labelled Bee Dataset (Sledevic, 2024).

A set of roughly 1000 frames from our ANU videos were selected and labelled by hand using Roboflow (Roboflow, 2025). These were selected from a range of short videos that we took with a phone camera. The videos were taken from different angles observing the hive entrance as bees flew in and out. See Figure 2 for an example frame from this dataset.

A further set of roughly 1000 frames from the Mendeley dataset were also selected to increase the dynamic contexts that the model would observe – different location, different beehive setup, different lighting, etc. Specifically, we used the image/label pairs found in '/detection/_bee_20230711a' of the dataset. See

Figure 3 for an example image from this dataset.

Figure 2: *Frame from one of our ANU dataset videos (authors').*

Figure 3: *Image from the Mendeley dataset (Sledevic, 2024).*

We then trained a YOLOv8 (Ultralytics, 2025) model on this dataset to obtain out object detection model. This performed fairly well, with a detection success rate of 88%. The full confusion matrix for the train/test/validate run can be seen in Figure 4. A key insight to notice is that this model is more prone to produce false positives rather than false negatives. This will be beneficial for us when we move to track the bees across frames, since we can filter out false positive detections much easier than we can recover from false negatives.

We also attempted to train a much larger model on around 4000 images rather than 2000, using additional labelled bee datasets online. However, this resulted in only very minor improvements to the false negative score, and actually made the false positive score worse. For this reason, we decided to stick with our 2000-image model for now.

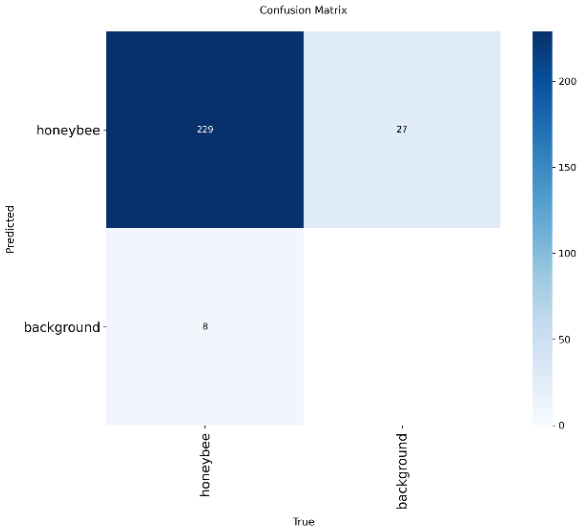


Figure 4: *Confusion matrix for YOLOv8 model.*

3 Problems with Existing Tracking Solutions

Once the object detection model was working, we started the rest of this project with high hopes. We knew that multi-object trackers (MOTs) have become quite common – and indeed, advanced – in the computer vision sphere, and we knew that there were many such off-the-shelf models to choose from.

However, we quickly noticed that bees are a fairly special use-case, and as such, the off-the-shelf models became quite inadequate for our task.

For starters, bees are small. Especially if you want to capture the whole hive entrance in a phone camera's field of view. On average, in the test videos we took, a single bee took up only 0.1% of any given frame. Secondly, bees are fast. A bee in one frame can move anywhere up to five times its body length in between frames. Furthermore, this can make them appear quite distorted in some frames due to the motion blur. A further and final complication is that bees change directions frequently and erratically. A bee flying forward can suddenly lift itself upwards and then dive bomb down again, all in the span of five or so frames.

All these factors combined make bees very hard to track with conventional off-the-shelf trackers. This is because many of these models

assume that the objects being tracked won't move much more than their length in between frames, with overlapping bounding boxes over time being the ideal scenario. This, of course, is not how bees behave in our videos.

We tried models like ByteTrack (Zhang et al. 2021), DeepSORT (Wojke et al. 2017), and inbuild YOLOv8 tracking (Ultralytics, 2024), but found that they would always result in constant identity splitting. That's where a bee in one frame may be labelled as 'bee 1', but then in the next frame, since it has moved so far away, the model assumes it's a new bee and assigns it the label 'bee 2'. This continues for most frames, so that a single bee can be assigned up to 10 or so tracks during its one flight (depending on the robustness of the model).

Other models rely on direction-based tracking, where they will predict the object's next position based on its motion so far, and then 'search' for that object in its expected direction in the next frame. If it doesn't find an object where it expects to – which happens often when bees change directions – then we once again get identity splitting.

A further approach we considered was to build and train a neural network to learn how to track bees itself. This, however, became quite complex and resource intensive to train. Instead, we opted for a new approach that we developed.

4 A New Model and Tracking Logic

This brings us to our model design, which combines aspects of current models with new innovations to develop a solution that is specifically suited to the bee-tracking problem.

4.1 Modelling track assignment as a conditionalized optimisation problem

We realised that instead of using the traditional tracking methods, we could simply model the tracking task as an optimisation problem. This is because we already know where the bees are – we're not trying to guess this. We just need to assign them to clusters, or tracks, over time.

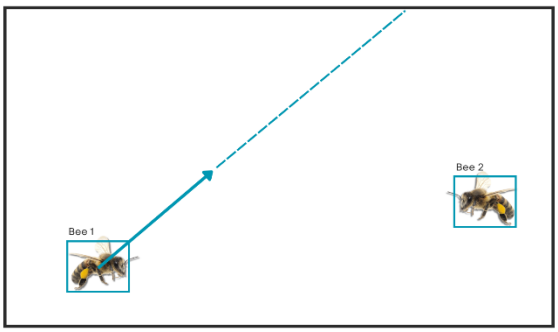


Figure 5: *Initial frame consisting of two bees. The directional expected direction of the left bee is considered.*

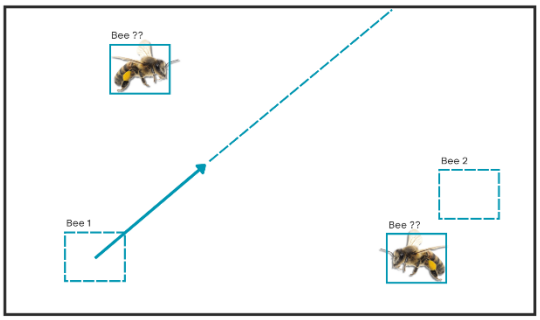


Figure 6: *Second frame showing the bees' new positions. Track assignments are yet to be made.*

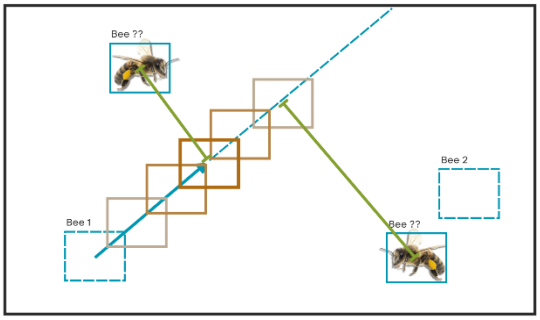


Figure 7: *Visual representation of the cost optimisation problem. The expected position of the track is shown in dark orange, with positions of less probability shown in lighter and lighter colours. The distance from each bee to the direction line are drawn in green.*

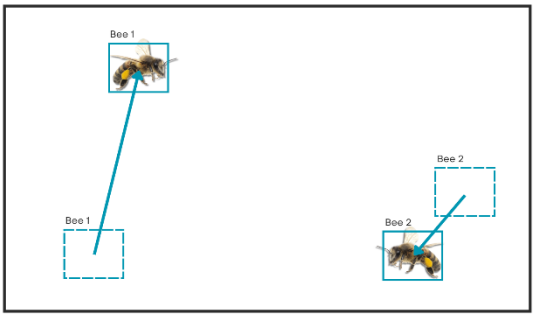


Figure 8: *Final bee-track assignments after the optimisation problem has been solved.*

Take for instance a case where we have two bees in an initial frame (as shown in Figure 5). Like direction-based object trackers, we can estimate the direction of the left bee, giving us its expected position in the next frame.

When we then step to the next frame, as shown in Figure 6, we still have two bees. However, now, we don't know which bee is which. What we do know, though, is the left bee's expected position, and so we can construct some probable locations for that bee along its directional line. As in Figure 7, the bee is most likely to be in the dark orange box, since that is our estimated position for it. It could, however, be slightly to either side of that box – shown in the lighter orange boxes. It could further still be slightly to either side of those – shown in the even lighter boxes – however, this is much less likely. Using this, we can then calculate the cost of assigning each bee to that track. The left bee is closer to the directional line, and is in-line with the dark orange box. The right bee, on the other hand, is further from the directional line, and is in-line with the much lighter box (see the light green distance lines in Figure 7). This brings us to the obvious conclusion (see Figure 8) that the left bee is in fact bee 1, and therefore the right bee is in fact bee 2.

This approach, however, is insufficient in itself to track bees. We further need to add conditional logic to force the model to work certain ways in certain conditions.

Both of these aspects of the model are explained in more detail in the following two sections.

4.2 The explicit optimisation problem (an application of linear sum assignment)

Since the general case of bee-to-track assignment assumes that we are matching a set of bees to an equal number of tracks, this problem can be modelled as a linear sum assignment problem (SciPy, 2008). This problem is then solved using the Jonker-Volgenant algorithm, a faster alternative to the famous Hungarian algorithm (Jonker & Volgenant, 1987).

We can construct a simple objective function, where we minimise the total cost of assigning bees, $j \in \{0 \rightarrow number\ of\ bees\ in\ frame\}$, to tracks, $i \in \{0 \rightarrow number\ of\ active\ tracks\}$:

$$Minimize \sum_{i=1}^{m} \sum_{j=1}^{n} c_{ij} \cdot x_{ij}$$

Where c_{ij} is the cost function, and x_{ij} is the decision variable to assign bee j to track i (x_{ij} must be either 0 or 1).

Cost function

Our cost function is a combination of two factors: the distance between the centre of a given bee's detection and the track of interest's expected position in a frame; and the angle penalty, which penalises bee-track assignments that have very different directions from the expected position:

$$c_{ij} = dist + angle\_penalty$$

As outlined in our tracking logic below in section 4.3, we further thresh out potential assignments by discarding assignments with cost greater than a given threshold, depending on the situation.

Distance component

As stated above, this is the distance between the centre of a given bee's detection (the 2D coordinate, d_j) and the track of interest's expected position in a frame ($expected_j$):

$$dist = ||d_j - expected_j||$$

We further define the expected position of track i as its position increment over the time between frames, Δt, from the track's last position, $last\_pos$, assuming a constant smoothed velocity, $v\_smoothed$:

$$expected_i = last\_pos_i + \Delta t \cdot v\_smoothed_i$$

Angle penalty

The angle penalty takes θ, the angle between the track's smoothed velocity vector and the vector drawn between the track's last position and the detection centre ($d_i - last\_pos_i$), and scales it by an appropriate factor to compare it with the distance in the cost function:

$$angle\_penalty = (1 - \cos\theta) \cdot 50$$

Adding this penalty mimics the probability weighting of the expected bee positions shown in Figure 5 in the previous section. Here, we simply do this by comparing the angle discrepancies along with the distance.

Velocity smoothing
To estimate the expected position of a track, we calculate its smoothed velocity. This takes the smoothed velocity from the previous frame, v_{old}, the difference between the track's last two consecutive points, Δpos, and a scaling factor, α (which we have set to 0.4):

$$v\_smoothed = \alpha \cdot \Delta pos + (1 - \alpha) \cdot v_{old}$$

Running this every frame for each track ensures that we are getting an informed updated estimate on where the track's next position should be. By smoothing the velocity out over time, we are reducing the risk that sharp local changes in bee velocity will disrupt the position estimate in the next frame.

Further notes on direction estimation
Initially when considering this tracker design, we relied on optical flow information to estimate the bee's direction of travel. Using a simple optical flow algorithm, we processed the video frame by frame to save the raw optical flow data in a local folder. We then sampled that data only within each of the bounding boxes and averaged all the direction vectors to obtain one 'direction vector' for that detection. We even added forwards and backwards flow calculation to give us a better estimate. However, this proved to be quite inaccurate for our use case. The optical flow vectors were often near zero, and even when they weren't, they were prone to point in random directions that did not line up with their actual direction of travel in the following frame.

RAFT (Teed & Deng, 2020), a heavy-weight optical flow model, did produce slightly better optical flow vectors. However, RAFT is highly resource intensive to run, and the marginal improvement it provided didn't make it worth the time cost.

This is why we decided to ditch the optical flow approach entirely, and simply rely on the average direction vector of the track as a simple and effective alternative.

It is further important to note that when a track has less than one point in it, and therefore has no way to calculate the average direction, that track's expected position is automatically set to its only detection's centre point. We then allow a higher cost threshold to assign detections to these tracks with only one point so far.

4.3 Our tracking logic

Arguably the most important aspect of this model, and indeed the project as a whole, is the conditional tracking logic that is applied alongside the optimisation problem. As we have previously addressed, bees are a very novel tracking case. We often end up with missed detections due to the speed and small size of the bees in the frame. We also constantly have bees entering and exiting the frame, often at the same time – whether that be to/from outside the field of view, or to/from the hive entrance.

Since our optimisation model relies on there being an equal number of bees to an equal number of active tracks, we need external logic that can handle the generation and removal of tracks to match the number of bees present in the frame. We also need a fallback system that can both accurately identify when we've missed a detection, and then robustly handle said missed detection without causing identity splitting or any further problems that will disrupt the counting.

This unique tracking logic that we developed is what we have laid out below.

Before we start, we need to initialise a few things: Firstly, throughout the entire video processing, we keep a list of all active tracks at any given time. Secondly, we explicitly start with zero detections and zero tracks (a pseudo frame 0 if you will). We then step through the video frame by frame, starting with frame 1, and apply the following logic each time:

If the current frame has an *equal number* of detections as the previous frame:

We assume that the same bee tracks are active (baseline case).

- ➔ All bee detections in this frame must be assigned to current active tracks.
- ➔ Only one track can be assigned per bee. No bee can be in multiple tracks at the same time.

We then check for any instances of ***fringe case 1*** (the case where we have a missed detection combined with new bee in this frame, which therefore mimics an equal number of bees in frame):

- ➔ If any assignment of a bee into a track results in a relatively large cost (greater than 300), then we assume that this bee should not belong to that track. Instead, we assume that this must be a new bee that has appeared at a different location. By reasoning of the equal detections case, this means that we have missed a detection.
- ➔ Assign this high-cost detection to a new track and proceed to the ***missed detections case***.

If the current frame has *n number more* detections than the previous frame:

We assume that *n* new bees have appeared.

- ➔ *n* new tracks must be started.
- ➔ The same track assigning rules as the equal number case apply (all bee detections in this frame must be assigned to current active tracks, and only one track can be assigned per bee).

We then check for any instances of ***fringe case 2*** (the case where the detection model splits one bee into two separate detections, a phenomenon we've called a *false splitting positive*):

- ➔ If any **new** detection in this frame is very close to or overlapping an existing detection (within 10 units), we assume that it's a false splitting positive.
- ➔ Do not count this detection for this frame (remove it from consideration), and re-run the frame again (check the number of detections compared to the previous frame and follow the logic from the start as usual, now with this detection removed).

If the current frame has *n number less* bee detections than the previous frame:

We assume that one or more tracks must have ended (meaning bees have left the frame – whether that's flying outside the field of view, or entering the hive and becoming obscured) and/or this frame is missing one or more detections.

- ➔ Here, we proceed to the ***missed detections case***.

Missed detections case:

Here, we have entered a state where our usual tracking logic cannot robustly handle the bee tracking task anymore. Therefore, for this frame, we will assign bees using a different, more specific logic – as follows:

- ➔ We know we have a list of active tracks at the moment of this frame. But one or more bees are missing from this frame.
- ➔ Therefore, we step to the next frame.
- ➔ In this new frame, we iterate through each *active track* in our list individually.
- ➔ For each track, we ask: Is there a detection in this frame that likely fits this track? We do this by checking if any detection has a low enough cost (less than 300) to be assigned to this track.
- ➔ If the answer is yes, then we temporarily assign that track ID to that detection. If the answer is no, then we don't assign this track ID to any detections.
- ➔ We do this process for each track. In this instance only, we allow detections to have multiple track IDs attached to

them. This stage is simply working out what detections could *possibly* belong to which tracks.

- ➔ We then take any detections in this frame that we have given multiple track ID assignments to, and we do a usual cost check to work out which assignments would reduce the overall cost for this frame. We then only assign one ID to each detection – the one that results in the lowest cost.
- ➔ If, after this process, there are any detections which don't have IDs assigned to them, we assume that these are new bees. We create new tracks for these detections.
- ➔ We then check if there are any active tracks that *don't* have any detections assigned to them in this frame. We add a counter variable to these tracks called a missed-count counter. If any track has a missed count of more than three, we deactivate that track.
- ➔ We then move to the next frame and resume our usual tracking logic from there.

It's important to note that the missed detections case is not as robust of a tracking logic compared to our usual logic when we use it on general non-missed detection cases. However, it performs very well for this specific instance where we have missed detections. That is why we only fall back to it when we're required to. In an ideal world, we wouldn't need to use this case at all. But given the imperfect nature of our object detection models, such cases are inevitable.

4 Solving the counting problem

We now had one last major problem to solve. Once we had robust bee tracking, we then needed to work out how to count them over time. We considered a number of solutions.

Firstly, we considered the simplest solution: we work out which direction the hive entrance is facing, and then determine the direction of 'in' and 'out' from that. For instance, say that the hive entrance faces to the left of the video. We could then define that tracks moving to the right over time are considered as going 'in', whereas tracks that move to the left over time are considered as going 'out'. This meant that we could simply compute the displacement of each track from start to finish and assign counts that way. However, this came with a number of problems. Firstly, bees do not always move in straightforward trajectories. They can zig-zag, hover up and down, and even fly in, bump their head against the hive wall, and then fly off. A simple left/right approach would not always accurately assign the bee's direction. The most prominent problem with this approach, however, is that despite our efforts to reduce identity splitting, it does still occur – especially for bees that fly out of the hive. These bees are much faster than the ones entering the hive, and so are often more distorted by motion blur. This can cause missed detections, and if there are too many missed detections in a row, even our tracking logic cannot pick them up, and will inevitably create two tracks for the same bee – one close to the hive, and one further out. If we apply this simplified counting logic, then this bee could be counted twice – which is exactly what we observed with this approach.

Our second, and much better, idea was to define a box around just the entrance slit. This way, we could define any track as 'in' if it starts outside the box and ends inside the box. Similarly, we could also define any tracks as 'out' if it starts inside the box and ends outside. All other tracks would not be counted. This solution worked a lot better for identity splitting – since any partial track segments that do not cross the box boundary do not count. We are only counting the number of tracks that cross that threshold. This did, however, come with its own challenges. Namely, how would we define the entrance box?

One solution to this is to get the user to draw an entrance box themselves by selecting four corner points on the first frame. The problem with this is that most of our videos are not stabilised, and camera motion will cause the box (which is defined in the fixed frame coordinates) to move away from the hive entrance. This allows bees to enter and exit

around the outside of the box, and we will not be able to count them.

To combat this, we implemented feature-mapping stabilisation of the entrance box. This would fix the box in place relative to its surrounding environment, so that even when the camera moved, the box would move with it. However, a big problem was this was that it relied on the user being very good at drawing an appropriate box. Testing during development showed that if they drew the box too large or too small, it would greatly affect the overall bee count.

We also considered another approach – what if we detected the entrance box instead? Using a new YOLOv8 model, this time detecting segmentation rather than bounding boxes (due to the oblique nature of the entrance box), we trained a model on roughly 100 images taken from our ANU dataset. We then needed to force the segment prediction to be a quadrilateral polygon, and use that as the entrance box on a frame-by-frame basis. This worked fairly well, and it meant that we didn't need to stabilise the box at all, since it was detected again each frame. However, our model would often miss part of the entrance box, and generate a polygon that was too small or rotated strangely. Again, this caused missed counts.

We finally settled on an intermediate solution. We run the entrance polygon detector on our video, sampling it every 20 frames. We then take all of those polygons and overlay them on top of the polygon on the first frame (this accounts for camera shake). We then average these polygons to find our 'average entrance polygon'. This is the polygon that we decide to use for the entire video, and we go back to stabilising it as before. This means that there is no need for user input. However, we understand that in some cases the entrance detector might fail in most frames, and therefore produce an average polygon that does not match the actual hive entrance. For this reason, we implemented a human-in-the-loop prompt into our workflow. Once the model had predicted an average polygon, it would show it on the first frame to the user. The user can then decide that they are happy with how this polygon looks, in which case it will be used, or they can choose to select their own, in which case we go back to the user-defined box. This provides a good balance of accuracy with fallbacks in case of failure.

The result of this process is shown in Figure 9 below.

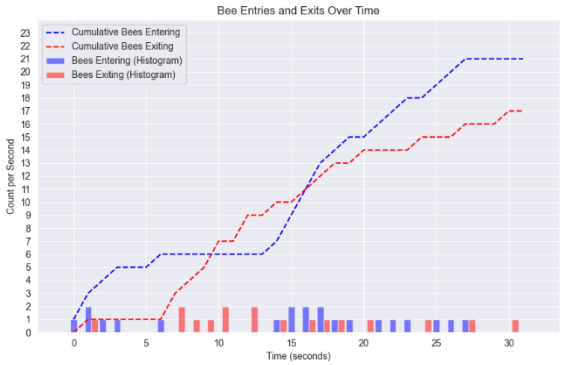


Figure 9: *Bee counts over time. Bees entering are shown in blue, and bees exiting are shown in red. The dotted lines show cumulative counts over time, whereas the bars below shown binned histogram counts for every one-second interval.*

4 App Deployment and Current Progress

Our initial aim in this project was to develop a phone app that could be distributed for citizen science use around the country. This would allow beekeepers to get immediate insights into the health of their hives, and would allow researchers to access a large pool of data about bee foraging behaviour across different contexts of climate, location, and weather events around the country. Due to time constraints that the unexpected difficulty of getting the basic model working put on us, we were not able to deliver this aspect of the project.

However, we have compiled the model into a desktop PC app that anyone can freely download and use if they wish.

The app consists of a simple user interface (GUI) that was built using python and Tkinter (Tkinter, 2001). It allows the user to select a video on their local disk for processing, to view the detection and tracking progress, and to then see the results displayed on the right-hand-side – as shown in Figure 10.

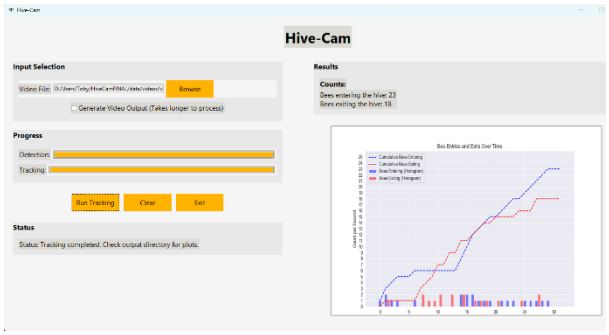


Figure 10: *Hive Cam App GUI.*

This app, along with all its code and resources is available to download on the GitHub project page linked at the top of this report.

5 Testing and Performance

Throughout development, we tested the performance of the model on two separate videos, which we named video A and video B. Both videos were 30 seconds in length, and were taken at the beehives on campus at the Australian National University (where our research is based). These beehives are made from polystyrene and have one long entrance slit on the font at the base.

We counted the number of bees entering and exiting the hive in each of these videos by eye to use as ground truth values. We then ran our detection, tracking, and counting model on them to get the model count values. These results are shown in Table 1.

As can be seen, our model performs fairly well across both test cases, with the 'entering' counts being only one less in both instances. Our 'exiting' counts are slightly worse, with a different of one in the case of video B, and a difference of three in the case of video A. As mentioned previously, this is due to the difficulty of reliably detecting – and therefore tracking and counting – bees that are exiting the hive. In some instances, we will get one or two detections inside the entrance box, but nothing after that. In other cases, we will only get detections once they've already left the entrance box. This means that these bees are not counted.

However, with current testing, although minimal, we are sitting at a 92% success rate for bee counting across entering and exiting counts.

This means that for a ten-minute-long video with the foraging activity of video B, we would miss 68 out of 860 bees.

Alternatively, for a ten-minute-long video with the foraging activity of video A, we would miss 27 out of 340 bees.

Test Video	Ground truth ENTERING	Ground truth EXITING	Model count ENTERING	Model count EXITING
Video B	*24*	19	*23*	18
Video A	*3*	14	*4*	11

Table 1: *Preliminary model testing to ground truth.*

6 Future Steps

There is still much work that can be done in the hive entrance monitoring space, and indeed still much that can be built onto our work on Hive Cam.

Firstly, there is ample room to add additional functionality to our model. Initially, we had hoped to develop a model that could distinguish between three classes of honeybees: regular, pollen-carrying, and drone. This would not be hard to add to our existing framework. One would simply need to train a new object detection model on a large and diverse enough dataset that labels these three classes. We could then plot these classes separately in the entering and exiting counts. This would allow researchers to understand more about the amount of food that is being brought into the hive in different seasons and climates. It would also allow beekeepers to identify when their drones are being ejected from the hive – something that is easy to miss without autonomous monitoring.

Secondly, more testing is needed of our current model to determine how effective it is when taken outside the ANU beehive context. It may prove that our detection models need to be

updates to be more robust to different hive styles and contexts.

Thirdly, there is the opportunity to develop this model into a citizen science app, as we had planned to do. This would allow our work to be used practically and create some meaningful change in the bee keeping and research communities. It is our plan to work on this in a subsequent project.

Another opportunity that we are keen to work in in a subsequent project is the development of a wholistic autonomous bee-monitoring system based in part on this model. While manually taking videos of bees is effective, it is not the ideal end-state for an autonomous monitor. We propose that more designs for true autonomy be explored – perhaps a product with fixed cameras that can feed the video on to either a local or central processing server for analysis, which can then forward count data to an app for immediate up-to-date info on hive health and activities.

7 Conclusion

This project has outlined the development of a robust bee-counting model using new innovations on previous tracking and counting strategies. We model the tracking problem as a conditionalized optimisation problem, enforcing comprehensive tracking logic onto a linear sum assignment problem. This allowed us to create the core building blocks of an autonomous hive entrance monitoring system, that could, with future work, allow beekeepers to get immediate information about the health of their hives, all by monitoring the least intrusive, and yet most telling part of the beehive.

8 Reference List

Dong, S. et al. 2023. "Social signal learning of the waggle dance in honey bees", *Science 379(6636), pp. 1015-1018.* DOI: 10.1126/science.ade1702. Available at https://www.science.org/doi/10.1126/science.ade1702.

Liang, C. et al. 2016. "Magnetic Sensing through the Abdomen of the Honey bee", *Sci rep 6(23657)*. DOI: 10.1038/srep23657. Available at https://pubmed.ncbi.nlm.nih.gov/27005398/.

Seeley, T. 2010. "Honeybee Democracy". *Princeton University Press*. ISBN: 9780691147215.

Pusceddu, M. et al. 2017. "Agonistic interactions between the honeybee (Apis mellifera ligustica) and the European wasp (Vespula germanica) reveal context-dependent defense strategies". DOI: 10.1371/journal.pone.0180278. Available at https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0180278.

Australian Honey Bee Industry Council (AHIC), 2022. "Who's who in the bee colony". Available at https://honeybee.org.au/wp-content/uploads/2022/04/220210-AHBIC-Fact-Sheet-Whos-who-in-the-bee-colony-APPROVED.pdf.

Chernak McElroy, S. 2016. "At the Hive Entrance: Look, Listen, Learn". *Keeping Backyard Bees*. Available at https://www.keepingbackyardbees.com/at-the-hive-entrance-look-listen-learn/.

Williams, M. 2025. "A Guide to Beehive Entrance Management". *The PerfectBee Blog*. Available https://www.perfectbee.com/your-beehive/starting-your-beehive/a-guide-to-beehive-entrance-management.

Pešović, U. et al. 2017. "Design and Implementation of Hardware Platform for Monitoring Honeybee Activity". Available at https://www.researchgate.net/publication/318505963_Design_and_Implementation_of_Hardware_Platform_for_Monitoring_Honeybee_Activity. Accessed 24 Oct. 25.

The Urban Beehive. 2025. "The Urban Beehive". Available at https://www.theurbanbeehive.com.au/. Accessed 24 Oct. 25.

Sledevik, T. 2024. "Labeled dataset for bee detection and direction estimation on beehive

landing boards". *Mendeley Data, V6*. Available at https://data.mendeley.com/datasets/8gb9r2yhfc/6. Accessed 24 Oct. 25.

Roboflow. 2025. "Roboflow". *Online software*. Available at https://roboflow.com/.

Zhang, Y. et al. 2021. "ByteTrack: Multi-Object Tracking by Associating Every Detection Box" Available at https://arxiv.org/abs/2110.06864. Accessed 24 Oct. 25.

Wojke, W. et al. 2017. "Simple Online and Realtime Tracking with a Deep Association Metric". Available at https://arxiv.org/abs/1703.07402. Accessed 24 Oct. 25.

Ultralytics, 2024. "Multi-Object Tracking with Ultralytics YOLO". Available at https://docs.ultralytics.com/modes/track/. Accessed 24 Oct. 25.

Ultralytics. 2025. "YOLOv8". Available at https://yolov8.com/.

Jonker, R. & Volgenant, A. 1987. "A Shortest Augmenting Path Algorithm for Dense and Sparse Linear Assignment Problems," *Computing 38, 325-340*.

SciPy. 2008. "linear sum assignment". Available at https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linear_sum_assignment.html. Accessed 22 Oct. 25.

Teed, Z. and Deng, J. 2020. "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow". Available at https://arxiv.org/abs/2003.12039. Accessed 24 Oct. 25.

Tkinter. 2001. "Tkinter". *Standard Python library.* Docs available at https://docs.python.org/3/library/tkinter.html.

Use of Generative AI

Both Grok and ChatGPT were used to generate most to all of the code that was used in this project. All code generated was based on explicit framework and logic provided by the authors, with the exception of an alternate missed-count handling solution that was proposed by Grok during code generation. Of course, this method ended up working better than our initial suggestion for this part of the logic.

No other instances of AI generated content were used in this project – including to write any part of this report.